Training–Rollout Mismatch in Low-Precision LLM Reinforcement Learning: A Survey

Zunhai Su
The University of Hong Kong

Abstract: Low-precision rollout generation can reduce execution cost while changing the numerical policy that supplies reinforcement-learning data. The resulting training–rollout mismatch links numerical approximation to gradient estimation, policy updates, and the trajectories encountered later in training. This survey reviews twenty focal studies through a common account of that feedback. We separate same-snapshot execution differences from policy staleness and decoding transformations, and distinguish conditional correction from response-level change of measure. A component-level taxonomy organizes numerical alignment and fidelity control alongside estimator correction, policy-update control, and schedule adaptation. We then compare the mechanisms these interventions target, the information they require, and the evidence supporting their learning and efficiency claims. Three findings recur: correction must be assessed over the learning trajectory; numerical coupling is meaningful at a specified execution interface; and the signed updates retained by an objective can matter more than an aggregate discrepancy statistic. Together, these findings position numerical fidelity and learning correction as complementary design resources. The survey closes with priorities in predictive diagnosis, method composition, stateful collection, and transferable evidence, supported by a versioned bibliography and an evolving public resource collection.

Keywords: quantized rollout; low-precision reinforcement learning; training–inference mismatch; importance sampling; quantization-aware training.

Literature snapshot: October 19, 2026.

Survey resources: https://github.com/ZunhaiSu/awesome-low-precision-rl

Living survey, under active development. Suggestions, corrections, and research collaborations are welcome via GitHub Issues.

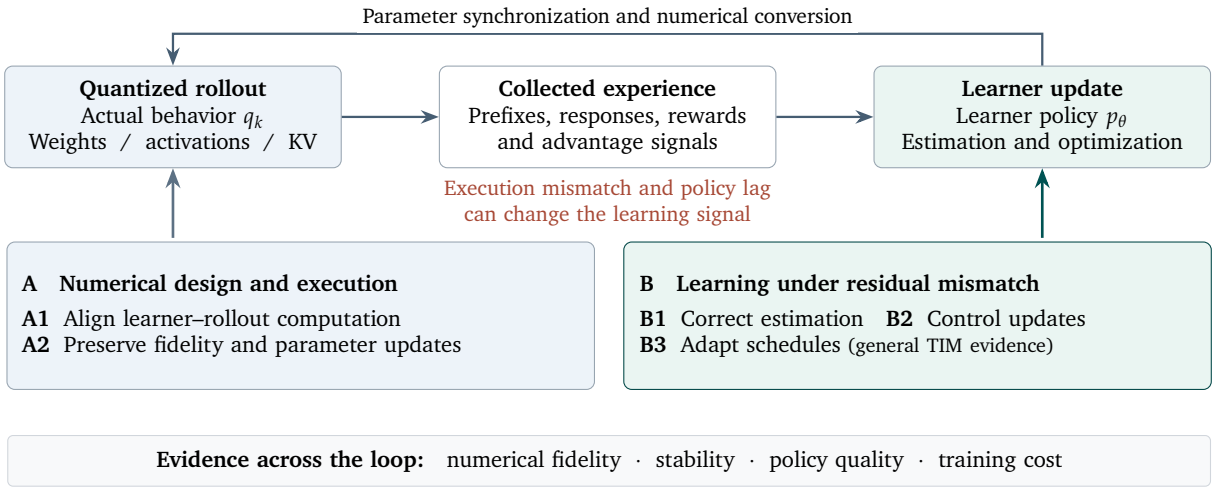


Figure 1: Survey at a glance. Numerical methods act on learner or sampler execution; learning methods change how samples drive updates. Their effects return through the rollout–learning loop.

Contents

1 Introduction and Scope **4**

1.1 Why Low-Precision Rollouts Complicate Policy Optimization 4

1.2 Scope, Related Surveys, and Review Protocol 4

1.3 Research Questions and Contributions 5

2 Problem Formulation and Taxonomy **6**

2.1 Learner, Behavior, Proximal, and Reference Policies 6

2.2 Execution Mismatch versus Policy Staleness 7

2.3 A Component-Level Taxonomy of Interventions 8

2.4 Configurations and Evidence Labels 9

3 Mechanisms: From Numerical Discrepancy to Learning Dynamics **9**

3.1 Sources and Propagation of Execution Discrepancy 9

3.2 Distorted Estimation and Update Decisions 10

3.3 Feedback across Optimization Steps 10

3.4 Diagnosing the Effects of Mismatch 11

4 Numerical Design and Execution Alignment **12**

4.1 Aligning Learner and Rollout Computation 12

4.2 Controlling Quantization Error and Preserving Parameter Updates 13

4.3 Residual Mismatch and Limits of Alignment 13

5 Learning under Residual Mismatch **14**

5.1 Correcting Gradient Estimation 14

5.2 Controlling the Influence of Policy Updates 15

5.3 Adapting the Optimization Schedule 16

5.4 Compatibility, Bias, and Failure Modes 16

6 Synthesis of Findings and Practical Trade-offs **16**

6.1 Coverage of Existing Studies 17

6.2 Findings on Stability and Learning Quality 17

6.3 Efficiency Gains and Implementation Costs 18

6.4 Limitations of Current Evidence 19

7 Open Problems **20**

7.1 Predicting Harmful Mismatch 20

7.2 Composing Numerical and Learning Interventions 21

7.3 Precision, Staleness, and Long-Horizon Interaction 21

7.4 Generalization and Comparable Evidence 21

8 Conclusion **22**

A Surveyed Papers and Section Mapping **23**

B Source Selection and Version Handling **24**

C Interpreting New Evidence **25**

1. Introduction and Scope

1.1. Why Low-Precision Rollouts Complicate Policy Optimization

In language-model reinforcement learning, the sampler is part of the learning algorithm: it determines the responses, rewards, and histories from which the next policy is learned. A numerical approximation therefore changes both an execution path and a source of training data. Its consequences can return through the next parameter update and the next round of generation. Quantized rollout makes this feedback explicit, although nominally identical precision formats can also produce different policies across training and serving engines.

The practical objective is an efficiently learned policy. Faster generation, closer probability agreement, stable optimization, and better task performance each describe a different part of that objective. An intervention may improve one while weakening another: for example, aggressive suppression can stabilize an update while removing useful learning signal. Understanding low-precision RL requires connecting these outcomes through the computation and estimator that produced them.

The recent literature approaches this question through several interfaces. FlashRL motivates correction using actual rollout probabilities; Jet-RL changes the numerical paths shared by learner and sampler; later studies intervene in quantization, clipping, gradient estimation, or local update dynamics [1, 3]. Their contributions are easiest to compare when the unit of analysis is the *method component*, rather than the paper title or precision format. An integrated system may contain both an execution intervention and an optimization intervention, neither of which should disappear behind a single label.

Figure 2 maps the surveyed work onto these intervention interfaces. It separates the numerical and learning branches, while showing how distinct components of an integrated method can occupy more than one class. Appendix A provides a corresponding paper-by-paper summary.

1.2. Scope, Related Surveys, and Review Protocol

Our focus is numerical and execution mismatch in LLM reinforcement learning, with emphasis on quantized rollout and closely related low-precision training configurations. We include asymmetric settings, such as a higher-precision learner with a lower-precision sampler, and settings in which both sides use low precision. Weight, activation, and key–value (KV) cache quantization are distinct numerical targets. Policy staleness is discussed where it interacts with execution mismatch, not treated as a synonym for it.

The review brings together twenty focal contributions: thirteen method or system studies, six studies that develop the formulation or analyze general mismatch, and one exploration-oriented boundary case. A supporting study can also provide direct low-precision evidence, as MiniRL does through its FP8-rollout experiments [14]. Appendix A maps these contributions to their discussions, with conference and preprint versions treated as one work.

The literature snapshot is October 19, 2026. We follow a narrative review approach, developing a curated collection through targeted checks of primary papers; selection and version handling are described in Appendix B. General post-training quantization, pretraining-only methods, and speculative-draft acceleration fall outside the focal scope unless they directly clarify the learner–sampler interface.

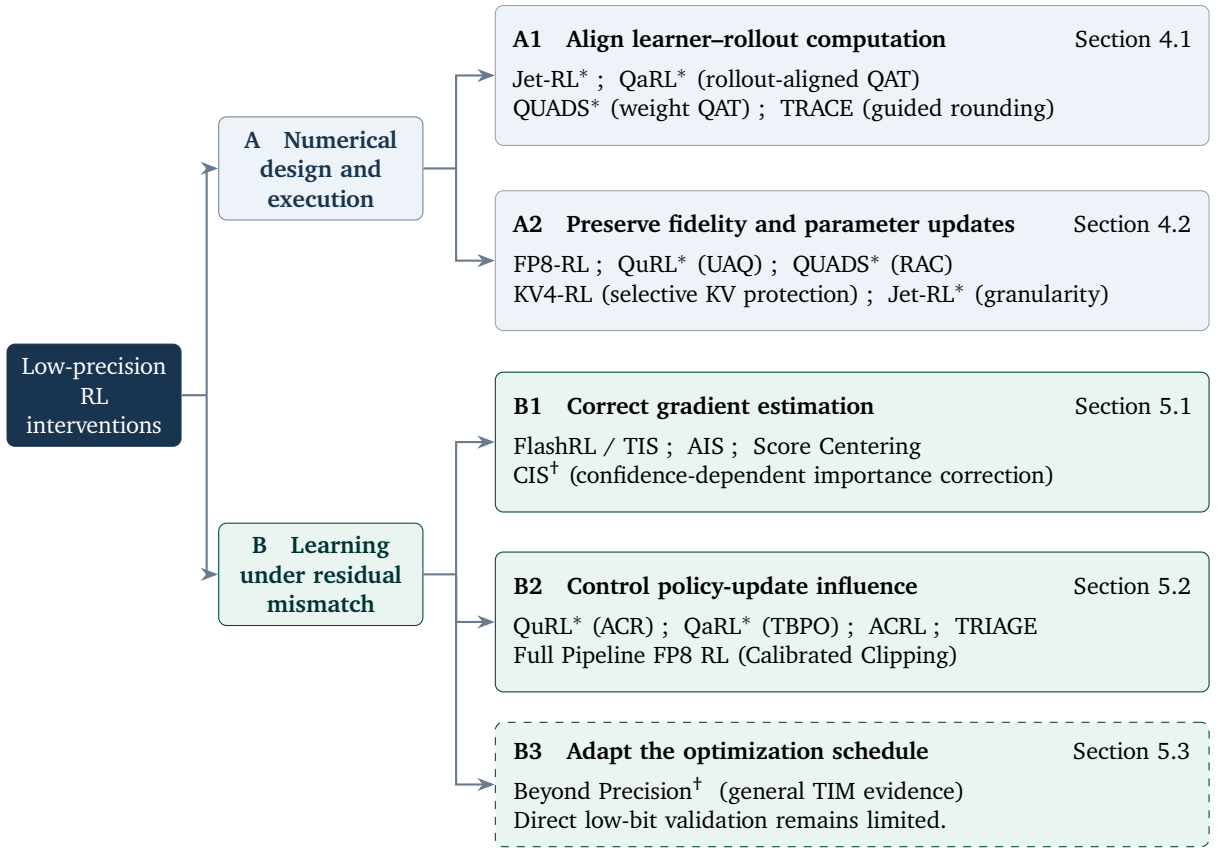


Figure 2: Taxonomy of the surveyed work. Branches identify what each component directly changes; repeated names reveal cross-category contributions. All thirteen core studies, six supporting studies, and the exploration boundary case are shown. Numerical configurations remain separate comparison axes. Appendix A gives the paper-by-paper summary and discussion locations.

Broader surveys cover low-precision LLM training, rollout strategies, and reinforcement learning for reasoning models [24–26]. Our emphasis is narrower: the relationship between actual sampling, numerical execution, and the update computed from sampled data. The exposition follows the component taxonomy developed below; numerical formats and experimental regimes are retained as separate comparison axes.

1.3. Research Questions and Contributions

Four questions guide the discussion: **RQ1**, how to separate execution mismatch from parameter changes and decoding differences; **RQ2**, how disagreement distorts estimation and optimization;

RQ3, which objects numerical and learning interventions modify; and **RQ4**, which findings hold under the reported experimental conditions and system costs.

Our contribution is to connect policy semantics, intervention mechanisms, and published evidence. Sections 2–3 establish a common notation and trace how execution differences enter learning (RQ1–RQ2). The component taxonomy then makes integrated methods comparable through the operations they change, organizing Sections 4–5 (RQ3). Section 6 relates the resulting learning outcomes to evaluation rules, information requirements, and cost (RQ4), leading to the research directions in Section 7.

2. Problem Formulation and Taxonomy

2.1. Learner, Behavior, Proximal, and Reference Policies

Let x be a prompt, $y_{1:T}$ a response, and $h_t = (x, y_{<t})$ its prefix. A policy is specified by both a parameter state and the computation used to evaluate it. Write $p_\theta(a \mid h)$ for the current learner policy and $p_k(a \mid h)$ for the same learner-side path at sampling snapshot θ_k.

The behavior policy $q_k(a \mid h)$ is the distribution that actually sampled the token. It includes the rollout engine, numerical representation, and sampling transformation. Temperature and vocabulary truncation therefore belong to its definition: a raw-model log-probability returned by an engine need not equal the probability after decoding transformations.

The notation conditions on a documented collection protocol. With deterministic execution and version schedules, sampler cache state is determined by the generated history, even when some cache entries were computed under earlier parameter versions. The subscript k abbreviates this specified rollout path; checkpoint identity alone is insufficient to reconstruct an unspecified stateful execution.

A proximal policy p_{prox} supplies the denominator of a policy-update constraint. A reference policy p_{ref} supplies a regularization target. These are different roles: a frozen pre-update learner may be proximal, while a much earlier policy is the KL reference. Neither is automatically the policy that generated every token. PPO and GRPO provide familiar contexts for these distinctions [21, 22]; mismatch-aware formulations require them to remain explicit [1, 14].

Table 1: Policies and quantities used throughout the survey. Execution paths are part of policy identity.

Symbol	Meaning
p_θ	Current learner policy under its specified execution path.
p_k	Learner-side policy evaluated at sampling snapshot θ_k.
q_k	Actual behavior policy that sampled the token.
p_{prox}	Policy used to define the proximal update ratio.
p_{ref}	Policy used for KL regularization; not the behavior denominator.
Δ_k	Same-snapshot log-probability difference, $\log p_k - \log q_k$.
d_θ	Current learner–behavior gap, $\log p_\theta - \log q_k$.
A_t	Advantage or other policy-gradient learning signal.
$s_\theta(a, h)$	Score vector, $\nabla_\theta \log p_\theta(a \mid h)$.

2.2. Execution Mismatch versus Policy Staleness

At a fixed prefix and token with positive relevant probabilities,

$$\frac{p_\theta(a \mid h)}{q_k(a \mid h)} = \underbrace{\frac{p_\theta(a \mid h)}{p_k(a \mid h)}}_{\text{parameter change}} \underbrace{\frac{p_k(a \mid h)}{q_k(a \mid h)}}_{\text{same-snapshot execution/decoding difference}}. \tag{1}$$

The factorization separates two comparisons at the same observed prefix. Its first factor measures parameter change since collection, including version lag in asynchronous execution; it requires no independence assumption. The snapshot policy p_k is a conceptual same-parameter reference; an implementation need not retain it explicitly to use a different proximal policy. The second factor isolates numerical execution when decoding rules are controlled, and includes a decoding component when temperature or top-p filtering differs. In partial rollouts, the sampling index may be token-dependent, $k = k(t)$. Moreover, carrying a cache across synchronization can preserve computations from earlier parameter states. In such an implementation, the current checkpoint index alone does not specify the behavior policy; cache provenance or the realized sampling probabilities are also needed. Cache retention is an implementation choice, so this qualification applies to implementations that carry those states across synchronization.

Two limiting cases make the distinction concrete. At the sampling snapshot, the learner-side parameter-change factor is one while execution mismatch can remain. With matched execution and decoding, the second factor can instead be one while policy lag remains. A numerical intervention and a staleness correction therefore solve different parts of the same ratio.

Probability ratios also require support conditions. A token observed from q_k has a defined denominator, but that alone does not establish that q_k covers every action with positive target probability. Truncation sampling can violate this condition. Similarly, a token-level change of measure is not automatically a correction for the prefix distribution. At fixed h,

$$\mathbb{E}_{a\sim q_k(\cdot|h)}\left[\frac{p_\theta(a \mid h)}{q_k(a \mid h)} f(a,h)\right] = \mathbb{E}_{a\sim p_\theta(\cdot|h)}[f(a,h)] \tag{2}$$

holds under support and integrability assumptions. It says nothing by itself about whether the outer distribution over h is the desired one. It also does not undo changes to group-relative advantages induced by collecting different responses.

For comparison, the full-response likelihood ratio is

$$W_\theta(y \mid x) = \frac{P_\theta(y \mid x)}{Q(y \mid x)} = \prod_{t=1}^{T} \frac{p_\theta(y_t \mid h_t)}{q_{k(t)}(y_t \mid h_t)}. \tag{3}$$

Here P_θ and Q are the induced response laws, and the denominator factors must be their actual conditional probabilities under the documented protocol. Unobserved execution randomness requires marginalizing that state or extending the likelihood to the corresponding execution events. The two response laws use a common stopping convention; an end-of-sequence decision is included when applicable. For a fixed behavior law and a parameter-independent reward R, the usual likelihood-ratio identity gives

$$\nabla_\theta \mathbb{E}_{P_\theta}[R] = \mathbb{E}_Q\left[W_\theta R \sum_{t=1}^{T} s_\theta(y_t, h_t)\right] \tag{4}$$

under support, integrability, and differentiation assumptions. The product illustrates why exact response-level correction can have high variance; multiplying only the current token ratio into an arbitrary sampled advantage is a different estimator. Nor does Equation (4) directly justify reusing group-normalized advantages computed under Q. Score Centering explicitly distinguishes sequence-level importance sampling from practical token-level corrections [10].

2.3. A Component-Level Taxonomy of Interventions

We group components by the object they directly modify. **A: numerical interventions** alter representation or execution. A1 matches computation between learner and sampler; A2 controls quantization error or preserves parameter updates. **B: learning interventions** alter how sampled data produce updates. B1 corrects gradient estimation; B2 controls the acceptance, magnitude, or direction of policy updates; B3 adapts the optimization schedule.

These are organizing categories, not disjoint mathematical sets. Their boundaries answer an operational question: if the component is removed while the rest of the system is held fixed, which computation or update rule changes? An estimator correction changes an update, and an execution alignment can improve quantization fidelity. The primary assignment follows the component's operative rule and design objective. In particular, a multiplicative weight is not sufficient to identify importance sampling. We distinguish a weight constructed to correct a distributional estimation problem from a weight constructed to control how an update changes disagreement. Schedule adaptation is an emerging direction whose direct low-bit evidence remains limited.

Table 2: Five intervention classes. These categories describe operative components, not exclusive paper types.

Class	Direct object	Representative components	Important residual question
A1	Shared computation	Precision flow; aligned QAT; guided rounding	Which interfaces remain unmatched?
A2	Quantization fidelity	UAQ; residual compensation; calibration; selective precision	Are useful parameter changes visible?
B1	Gradient estimator	TIS; AIS; CIS; Score Centering	Which expectation is corrected, and with what bias?
B2	Update influence	ACR; TBPO; calibrated clipping; direction-aware controls	Which updates are retained or suppressed?
B3	Optimization schedule	Mismatch-informed LR adaptation	Is stability obtained at comparable learning progress?

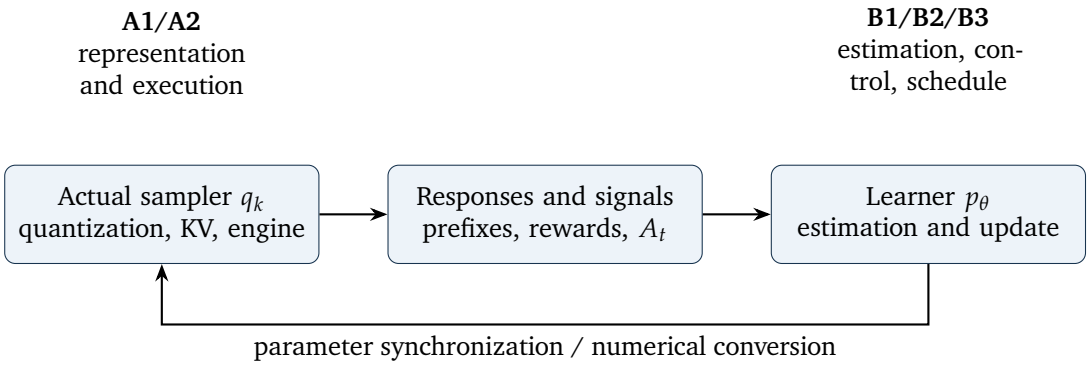


Figure 3: The feedback loop reviewed in this survey. Numerical interventions may affect either execution endpoint; learning interventions act on the update. Arrows express data flow, not a claim that interventions must be applied in a particular order.

2.4. Configurations and Evidence Labels

The same intervention can operate under different numerical configurations. W/A/KV precision, dense versus mixture-of-experts (MoE) architecture, low-rank versus full-parameter updates, synchronous versus partial collection, and native kernels versus quantize–dequantize simulation are recorded separately. "FP4 training" does not specify which operations use FP4 or whether master weights, backward computation, and optimizer states remain at higher precision.

Contribution and evidence also deserve separate treatment. A formulation study can contain direct quantized-rollout experiments, while a quantized system can adopt an existing correction. Keeping those roles explicit preserves both relevant experiments and accurate attribution.

3. Mechanisms: From Numerical Discrepancy to Learning Dynamics

3.1. Sources and Propagation of Execution Discrepancy

A numerical policy is determined by more than its checkpoint. Scale selection, rounding, accumulation, kernel choice, batch shape, and the treatment of cached states can alter the output distribution. Sharing a nominal format therefore does not imply sharing a computation. FP16-based mismatch analysis provides a useful precision-selection comparison, while R3 exposes the importance of matching discrete routing decisions in MoE execution [15, 16]. Neither should be treated as evidence that every routing discrepancy originates in quantization.

Two measurement settings must remain separate. *Shared-prefix comparison* holds the token history fixed and compares conditional distributions. *Free-running comparison* allows sampling differences to change the histories visited. The first is valuable for isolating a numerical interface; the second measures behavior after divergence has propagated through generation. A favorable shared-prefix statistic is not, on its own, a guarantee of similar trajectory-level learning signals.

Discretization introduces a further asymmetry. An update can be visible in the learner's parameter space but fail to change the sampler's quantized representation. Conversely, a small change near a rounding boundary can abruptly change a codeword. QuRL studies the former through update-aware quantization; TRACE targets inconsistency in rounding outcomes [5, 13]. The relevant object

is therefore not only error magnitude, but its relationship to the evolving parameters and the discretization boundaries.

3.2. Distorted Estimation and Update Decisions

At an observed prefix, numerical disagreement can change an update through its weight, its expected direction, or the clipping branch it activates. These mechanisms require different diagnostics. A probability difference and an importance ratio, for example, encode different information. Writing a learner probability as $p = q + \delta$ gives $p/q = 1 + \delta/q$: the same absolute perturbation has a larger relative effect when q is small. This elementary observation motivates reporting tail behavior and confidence-conditioned statistics rather than only mean absolute probability error. CIS makes confidence dependence explicit in its correction design [20].

Estimation can also acquire a systematic drift. For a normalized differentiable learner distribution, $\mathbb{E}_{a\sim p_\theta}[s_\theta(a,h)] = 0$. Under behavior sampling, the corresponding expectation need not vanish. At fixed h,

$$\mathbb{E}_q[As_\theta] = \mathrm{Cov}_q(A, s_\theta) + \mathbb{E}_q[A]\mathbb{E}_q[s_\theta]. \tag{5}$$

The expectations are conditional on the prefix and include the sampled action and any random continuation used to form A. Centering advantages across a prompt group need not center them at each prefix. Score Centering uses this distinction to motivate an additive correction [10]: it removes a first-moment drift term while leaving the behavior distribution itself unchanged.

Mismatch additionally affects decisions made by a surrogate objective. In a clipped policy-gradient objective, changing a ratio can change which branch is active and therefore whether a token contributes gradient. Thus, a perturbation can matter through a threshold crossing even when its absolute magnitude appears modest. The full-pipeline FP8 study isolates a failure involving excessive clipping of negative-advantage tokens [11]. This is a different intervention target from merely decreasing the variance of a sampled importance weight.

The sign dependence can be seen without assuming a particular quantizer. Let $\ell(r, A) = \min\{rA, \mathrm{clip}(r, 1-\epsilon, 1+\epsilon)A\}$, with $0 < \epsilon < 1$ and fixed A. Away from the two boundaries,

$$\frac{\partial \ell}{\partial r} = \begin{cases} A\,\mathbf{1}\{r < 1+\epsilon\}, & A > 0, \\ A\,\mathbf{1}\{r > 1-\epsilon\}, & A < 0, \\ 0, & A = 0. \end{cases} \tag{6}$$

Thus the active clipping side depends on the learning signal. For an illustrative objective that clips the combined learner–behavior ratio, a same-snapshot ratio of 0.7 with $\epsilon = 0.2$ already suppresses a negative-advantage token, even before a learner update. This is not the decoupled surrogate in Section 5.1; the example explains why naming the clipped ratio is essential to interpreting a failure or a repair.

Figure 4 makes this sign dependence explicit for the standard surrogate. It also shows why clipping the combined learner–behavior ratio can suppress an update that a proximal ratio would still permit.

3.3. Feedback across Optimization Steps

The learning loop turns a local discrepancy into a dynamic process. A sampled response changes the learner; the learner is synchronized back to the sampler; the next rollout is then generated from an

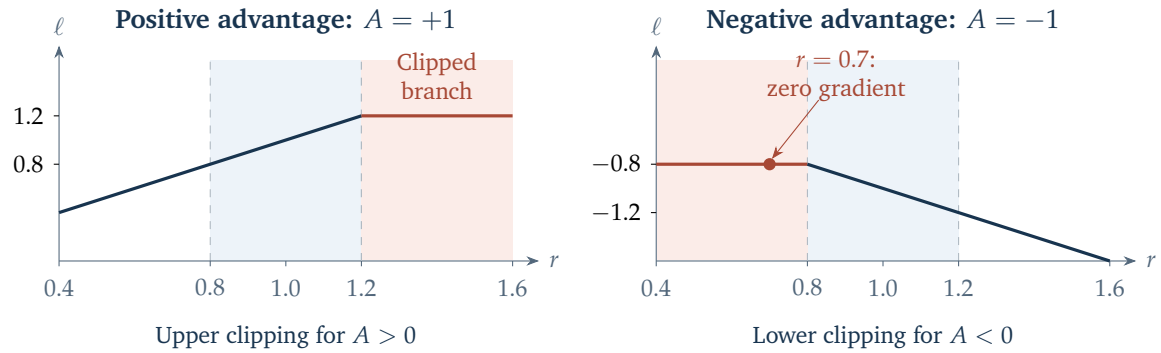


Figure 4: Analytical illustration of Equation (6) with $\epsilon = 0.2$ and $A = \pm 1$. Blue shading marks $[0.8, 1.2]$; red shading marks the flat, clipped branch. A ratio of 0.7 lies on the zero-gradient branch for negative advantage. The relevant interpretation depends on which policy ratio the objective clips.

updated numerical realization. Stability depends on this feedback, not solely on whether one frozen batch has a small error.

Direction matters because two updates with the same gap magnitude can move learner and behavior probabilities differently. In a simplified local picture, reducing a learner probability that is already below the behavior probability can enlarge their log gap. ACRL and TRIAGE use update-direction information in their controls [9, 12]. For a shared-parameter network, however, one token's update changes other probabilities too. The sign argument supplies local intuition; interactions through shared parameters determine the scope of any stability result.

Optimization scale adds another feedback channel. Beyond Precision studies the relationship between mismatch and optimization dynamics and proposes response-length-triggered learning-rate decay [18]. This connects numerical observations to the scale of optimization, while leaving transfer of the schedule to aggressive FP4 execution open.

3.4. Diagnosing the Effects of Mismatch

Diagnosis begins with the comparison used to isolate an effect. Replacing an inference engine can simultaneously change arithmetic, batching, memory use, and generated trajectories. A resulting quality difference cannot automatically be assigned to the precision format. VeXact constructs a zero-mismatch reference to help separate execution mismatch from other training effects [19]. It supplies a diagnostic counterfactual against which the effects of accelerated execution can be interpreted.

The evidence chain has three links: an execution change alters conditional probabilities; those probabilities alter the estimator or update decision; repeated updates alter the learned policy. Shared-prefix replay can isolate the first link, update diagnostics probe the second, and training trajectories establish the third under a stated horizon. An ablation that identifies one link is valuable, but leaves the others to be established. Section 6 follows these links when drawing together the published findings.

Box 1. Does smaller mismatch always mean better learning?

QeRL combines low-precision execution, low-rank adaptation, and adaptive quantization noise, making exploration an explicit part of its setting [2]. It is a boundary case for this survey, not a method for eliminating mismatch. Its results motivate separating numerical agreement from task utility, but do not imply that arbitrary cross-engine error is beneficial. A useful comparison must ask what changed in the sampling distribution, what adaptation capacity the learner had, and whether the gain persists under matched evaluation.

Synthesis. Mismatch is not one scalar defect. Conditional probability error, trajectory selection, estimator drift, threshold decisions, and cross-step feedback are distinct objects. A method should be assessed against the object it actually changes.

4. Numerical Design and Execution Alignment

Numerical design offers two complementary ways to change the policy that generates training data. Alignment brings learner and sampler computations closer at a shared interface; fidelity control preserves the information that a low-precision representation would otherwise lose. The distinction matters because two approximate paths can agree without either becoming more accurate relative to a higher-precision model.

4.1. Aligning Learner and Rollout Computation

Jet-RL moves the discussion from matching bit widths to matching precision flow [3]. Its unified FP8 design coordinates the numerical paths and quantization granularity used by learner and sampler. Agreement is pursued through corresponding operators, rather than inferred from an FP8 label alone. This path-level view provides a starting point for later approaches that couple the endpoints more selectively.

QaRL implements such coupling through rollout-aligned quantization-aware training (QAT): the learner's forward pass reflects the representation used during generation [6]. QUADS takes a deliberately asymmetric approach, fake-quantizing learner weights while retaining higher-precision learner activations [8]. Both designs bring part of the learner computation closer to the sampler, but QUADS leaves activation error to a separate rollout-side component. Their comparison reveals that alignment can be selective; uniform precision across the whole model is neither its definition nor a prerequisite. QAT emulation and native low-precision kernels nevertheless remain distinct numerical paths.

TRACE goes further by sharing realized quantization information [13]. It records compact rollout-side information to guide learner-side FP4 rounding, addressing cases in which a common quantizer specification still produces different decisions near a rounding boundary. Its local rounding argument explains this interface-level benefit without establishing monotonic reduction of full-network policy divergence.

These approaches couple the endpoints through increasingly specific information: a precision-flow design, an emulated representation, or realized numerical decisions. They represent different choices about what to share, rather than successive points on a universal performance ranking. Routing replay follows a related principle for discrete MoE execution choices [15]. As coupling becomes

more specific, correspondence between token positions, modules, and parameter versions becomes part of the method's validity. The remaining computation still determines how closely the complete policies agree.

4.2. Controlling Quantization Error and Preserving Parameter Updates

Alignment alone does not ensure that the sampler can express the changes made by training. For an illustrative deterministic quantizer Q, an update can satisfy

$$Q(w + \delta w) = Q(w), \tag{7}$$

even when $\delta w \neq 0$. Visibility depends on the distance to a quantization boundary and on scale behavior, as well as update magnitude. QuRL's Update-Aware Quantization uses invariant scaling to alter the relationship between quantization resolution and parameter updates [5]. Its numerical contribution is therefore distinct from ACR, which modifies the clipping rule in Section 5.2.

QUADS addresses another residual: activation error in the rollout path. Its Residual Activation Compensation complements learner-side weight QAT, allowing alignment and error reduction to operate on different parts of the system [8]. The component ablations are particularly useful here because they distinguish what weight alignment recovers from what activation compensation adds.

Maintaining fidelity also requires following a changing policy. FP8-RL integrates blockwise W8A8 rollout, scale recalibration for FP8 attention and KV execution, and dynamic weight synchronization [4]. The contribution lies in keeping these numerical components operational throughout RL, together with adopted TIS/MIS correction. Its v2 experiments also explore end-to-end FP8, making learner precision an experiment-level attribute rather than a fixed description of the entire paper.

KV4-RL allocates fidelity across positions instead of raising precision uniformly [17]. INT4 KV storage is supplemented by bounded INT8 protection for attention-sink and recent positions. Its retained TIS recipe makes the distinction between sampler fidelity and conditional correction especially clear: protecting the cache changes the data-generating computation even when the correction is held fixed. The experiments use colocated, quota-based partial collection, with generation and optimization alternating; responses may span collection boundaries.

Across these studies, the useful precision budget is the one that preserves information relevant to learning. Parameter rescaling protects update visibility, residual compensation addresses activation error, recalibration follows distributional change, and positional protection targets sensitive cache states. W/A/KV identifies where these mechanisms operate; it does not replace the distinction between them.

4.3. Residual Mismatch and Limits of Alignment

The scope of alignment follows its interface. Matching weights can leave activation differences; matching a quantizer can leave different surrounding kernels; protecting selected states can leave the rest of the network unchanged. Agreement and fidelity are therefore separate achievements, each tied to a parameter snapshot and input history.

This partial coverage explains why numerical methods often coexist with learning correction. A more faithful sampler can still lag behind the learner, while a closely matched conditional distribu-

tion cannot undo the histories excluded by earlier sampling decisions. The residual determines what a learning-side intervention has left to address.

Synthesis. Numerical methods differ in what they share and what they preserve. Their effect on learning depends on the protected interface, the remaining discrepancy, and the cost of maintaining that relationship.

5. Learning under Residual Mismatch

Learning interventions accept that the rollout data and learner computation may differ. Their goals range from adjusting an estimator to controlling harmful updates or changing the optimization schedule. They should not all be evaluated as approximations to the same unbiased gradient: some deliberately modify the update to trade bias, variance, or stability against one another.

5.1. Correcting Gradient Estimation

Multiplicative correction and truncation. Importance sampling supplies a conditional change of measure under the assumptions of Equation (2). Practical surrogate objectives usually distinguish the mismatch correction from the proximal update ratio. With frozen p_{prox} and recorded behavior q, define

$$c_t = \frac{p_{\text{prox}}(a_t \mid h_t)}{q(a_t \mid h_t)}, \qquad r_t(\theta) = \frac{p_\theta(a_t \mid h_t)}{p_{\text{prox}}(a_t \mid h_t)}. \tag{8}$$

A representative decoupled surrogate is

$$\widehat{L}(\theta) = \frac{1}{N} \sum_t \text{sg}(\bar{c}_t) \min[r_t(\theta) A_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) A_t], \tag{9}$$

where $\bar{c}_t$ is a possibly truncated correction, the sampled advantages are held fixed, and sg denotes no differentiation through the correction weight. This is a schematic comparison device, not a claim that every reviewed method implements Equation (9) identically. It separates two operations: bounding the correction c_t and clipping the proximal ratio r_t. An upper-truncated correction, for example, takes $\bar{c}_t = \min(c_t, C)$, whereas masking can discard selected ratios. Here N counts valid sampled tokens; response-level averaging or length normalization changes their relative weights and must be retained when comparing implementations.

FlashRL's TIS provides a basic bounded correction using actual rollout probabilities [1]. MiniRL exemplifies the mismatch-aware distinction between behavior and proximal policies underlying this notation [14]. FP8-RL adopts token-level TIS/MIS variants, while KV4-RL retains TIS alongside cache protection [4, 17]. These uses contribute system evidence without creating additional estimator families. Truncation limits extreme influence but changes the expectation. A stable truncated objective should therefore not be described as exact importance sampling merely because its denominator comes from the sampler.

Adapting the amount and location of correction. AIS uses observable mismatch diagnostics to interpolate between uncorrected and fully importance-weighted gradients [7]. Its adaptive quantity is the estimator mixture, not a precision schedule. CIS instead derives a confidence-dependent

importance-weight cap from a discrepancy representation [20]. These approaches differ in what they adapt: overall correction strength versus the location of truncation across token confidence. CIS is used here as general TIM evidence; its BF16 cross-engine experiments are not direct FP4 validation.

Additive score correction. Score Centering replaces the score at a prefix with

$$\widetilde{s}_\theta(a,h) = s_\theta(a,h) - \mathbb{E}_{b\sim q(\cdot|h)}[s_\theta(b,h)]. \tag{10}$$

With the exact expectation, its mean under q vanishes and the drift term in Equation (5) is removed [10]. The remaining covariance is still measured under q, so this is not full recovery of an on-policy gradient. The reported implementation uses sampler top-k log-probabilities and models the omitted tail using the learner distribution; its default is $k = 128$ [10]. The resulting approximation is more specific than simply discarding the tail. Its coefficients are detached when forming the correction, and its fidelity depends on the reconstructed behavior distribution.

The distinction between multiplicative and additive correction also determines the information returned by the sampler. Ratio-based methods need probabilities of sampled actions; exact score centering needs a behavior-weighted score expectation. Truncation and reconstruction make different approximations to these quantities. Their derivative interpretation additionally depends on the learner computation, including any surrogate gradient through a quantizer.

5.2. Controlling the Influence of Policy Updates

Restoring or reshaping update constraints. QuRL's Adaptive Clipping Range adjusts the update boundary in response to the relationship between proximal and behavior probabilities [5]. QaRL's TBPO combines negative-sample treatment with sequence-level trust-band control [6]. The full-pipeline FP8 method first matches the lower clipping quantile on negative-advantage tokens to a BF16 reference, then adjusts the upper boundary to rebalance positive and negative contributions [11]. Periodic reference passes amortize the calibration cost. All three act on the update constraint, but use different signals and aggregation levels. "Calibration" in the last method refers to objective boundaries, not quantizer scales.

The shared issue is which learning signal reaches the parameter update. A token-level boundary can restore a contribution lost to clipping, whereas a sequence-level trust band controls a whole response. Widening either region admits more updates, but their value depends on advantage sign and influence. The clipping count becomes informative when connected to those retained contributions.

Controlling disagreement through update direction. ACRL uses advantage-guided probability adjustment, with response-level discrepancy relative to a reference controlling token reweighting [9]. Its objective is regulation, not unconditional minimization of discrepancy. TRIAGE instead combines segment-local risk information, direction-selective gating, rebalancing, and bounded repair in native NVFP4 execution [12]. Their common placement in B2 reflects the role of the update rule, not equivalence of their risk models or experimental settings.

The granularity of control matters. A response-level statistic aggregates evidence across many tokens; a segment statistic can expose local concentration; a token rule is directly selective but may

be sensitive to noise. Aggregation trades localization against statistical reliability. Finer granularity improves localization only when the resulting diagnostic remains reliable enough to guide the update.

5.3. Adapting the Optimization Schedule

A third intervention changes the scale of learning over time. Beyond Precision motivates learning-rate scheduling through the evolution of optimization and mismatch [18]. This is complementary to changing a numerical path or a token weight: it modifies the trajectory of parameter updates themselves.

Schedule adaptation remains less developed for direct low-bit settings than the estimator and update controls above. Its promise depends on preserving progress as well as stability: smaller steps can avoid divergence while delaying useful learning. Comparable quality within a comparable budget is therefore the relevant test of transfer to aggressive quantization.

5.4. Compatibility, Bias, and Failure Modes

Several designs already combine numerical and learning components. QaRL, QuRL, and QUADS provide evidence for the combinations studied within each system; Score Centering describes composition with importance weighting by centering the weighted score [5, 6, 8, 10]. These results motivate composition while leaving its behavior under other combinations open.

Compatibility depends on more than category membership. Components must refer to the same behavior policy and parameter snapshots, act coherently on the target estimator, and obtain their required information at acceptable cost. A centering term for one estimator need not center a differently weighted score; successive masks can remove the same examples; a reference pass can change the cost of an otherwise inexpensive rule. Section 6 relates these dependencies to the published system evidence.

The residual error helps explain where composition has value. Conditional correction leaves the collected histories unchanged, numerical alignment leaves policy lag, and clipping controls influence without reproducing the sampler's arithmetic. Combining components is useful when their remaining errors complement one another and the joint rule preserves enough signal to learn.

Synthesis. Estimator correction changes an expectation; update control changes which contributions are allowed to act; scheduling changes their evolution over training. Their composition depends on shared probability semantics and the learning signal that survives.

6. Synthesis of Findings and Practical Trade-offs

The literature offers its clearest comparisons through controlled changes within a system. Read together, these studies reveal how numerical fidelity, estimator behavior, and update selection interact. The synthesis below follows those connections, relating learning outcomes to the information and execution cost required to obtain them.

6.1. Coverage of Existing Studies

The corpus spans several configurations that should remain distinct. Asymmetric learner–sampler precision emphasizes a gap between numerical representations. Low precision on both sides emphasizes the consistency of their execution paths and the behavior of the resulting update. KV-focused work changes a state-storage interface, while weight or activation quantization changes other parts of computation. Low-rank adaptation and full-parameter RL additionally differ in which degrees of freedom can respond to quantization.

The autoregressive equations in Sections 2–5 describe token-generation paths. AIS also evaluates diffusion-based LLaDA alongside autoregressive Qwen models [7]. Its denoising experiments extend the empirical scope of the source, but require a generation-process-specific likelihood interpretation; they are not additional validation of the autoregressive factorization as written here. We use its autoregressive evidence for direct comparison in that formulation.

Architecture and collection protocol further condition conclusions. MoE execution introduces discrete route choices; cache-heavy long responses expose different runtime bottlenecks from shorter generations; partial collection permits a response to contain tokens sampled under different parameter versions. A method evaluated in one of these regimes may motivate a hypothesis in another, but that transfer is not itself an experimental result.

Appendix A maps the focal studies to their detailed discussions. The companion catalog records numerical configurations and source versions, while the evidence file links observations from all thirteen core method/system studies to the relevant source sections. Table 3 draws out six comparisons that most directly illuminate the findings below.

6.2. Findings on Stability and Learning Quality

Three findings connect studies that otherwise differ in format, architecture, and correction rule.

Correction must be evaluated over the learning trajectory. QuRL shows why an initially stable correction can be an incomplete account of a run: disagreement grows over a longer horizon, while ACR and UAQ address different changes in clipping and update visibility [5]. AIS adapts the correction mixture, ACRL regulates discrepancy through update influence, and Score Centering targets an estimator drift term [7, 9, 10]. The common lesson is temporal: the distribution, correction, and learner evolve together. Early agreement or bounded weights describe only part of that evolution; the resulting update and quality trajectory establish whether the intervention remains useful.

Numerical coupling is useful at a specified interface. Jet-RL couples precision flow, while QaRL, QUADS, and TRACE couple representations or realized rounding decisions [3, 6, 8, 13]. QUADS gives a particularly informative component comparison: asymmetric weight QAT recovers much of the learning loss, and rollout-side residual compensation improves the remaining gap. Its asymmetric-versus-symmetric QAT comparison further separates nominal symmetry from effective alignment. FP8-RL adds the system perspective, showing how calibration and synchronization support a changing low-precision policy [4]. Together, these studies locate the benefit in a specified interface. Transfer depends on whether another stack preserves that interface and the surrounding computation.

Which updates survive can matter as much as average disagreement. The full-pipeline FP8 study connects excessive lower-side clipping of negative-advantage tokens to pathological training dynamics, whereas TRIAGE uses local and direction-dependent information to select interventions [11, 12]. Both connect stability to the signed learning signal that remains after control, revealing structure hidden by an aggregate clipping fraction or mean log-probability gap. TRIAGE also limits its token-wise directional predictiveness in its MoE routing-conditioned analysis, underscoring the difference between a population pattern and a reliable predictor for each token.

Table 3: Selected published observations and their interpretive limits. These within-paper comparisons illustrate the synthesis; the twenty-work coverage map appears in Appendix A. No row defines a performance ranking across papers.

Study / setting	Published comparison or observation	What the evidence supports
QuRL [5] INT8/FP8 rollout	Longer-run behavior differs from early TIS behavior; ACR and invariant-scaling ablations address distinct failure modes.	Correction magnitude, clipping semantics, and update visibility are separate concerns; a short stable window is insufficient.
QUADS [8] NVFP4 MoE rollout	Weight-only learner QAT recovers much of the loss; adding rollout activation compensation further improves held-out scores.	The two components have complementary evidence within this design; neither universal necessity nor cross-engine transfer follows.
Score Centering [10] BF16 learner; quantization or lag	Shared-objective experiments compare additive centering and importance correction; their composition helps in the reported staleness tests.	Estimator drift and importance-weight control are distinguishable. The implemented behavior expectation is approximated.
Full Pipeline FP8 [11] FP8 learner and rollout	Calibrated Clipping improves the reported learning outcomes across scaling choices. Main GRPO results use best-checkpoint selection over 500 steps.	Restoring signed update contributions can matter; the selected endpoint is not a fixed-budget final-checkpoint comparison.
TRIAGE [12] Native NVFP4	Local tail patterns motivate direction-selective gating and repair. Token-wise predictiveness has a narrower scope than the population characterization.	Local and signed diagnostics can be informative, with limits under MoE coupling; token-independent contraction is not established.
KV4-RL [17] INT4 KV; partial collection	Unprotected INT4 KV misses later learning gains under the retained TIS recipe; bounded INT8 protection recovers near-BF16 endpoint quality.	Conditional correction and cache fidelity address different effects. Timing and quality claims apply to the studied collection workload.

These findings connect fidelity and correction through the learning signal they preserve. Better execution can change which tokens require correction; a better update rule can make residual execution error tolerable. The strongest evidence links a component ablation to both the proposed mechanism and a learning outcome. This connection is what turns numerical agreement into a claim about effective training.

6.3. Efficiency Gains and Implementation Costs

The same complementarity has a system cost. For serialized phases, iteration time can be decomposed as

$$T_{\text{iter}} = T_{\text{rollout}} + T_{\text{learner}} + T_{\text{sync}} + T_{\text{quant/calib}} + T_{\text{other}}, \tag{11}$$

where synchronization and quantization/calibration costs are separated from generation and learning. Accelerating one phase exposes the others. With overlapping execution, the relevant quantity is the critical path, so the phase times in Equation (11) cannot simply be added.

Each intervention changes a different part of this cost. FP8-RL illustrates the work needed for scale estimation and numerical conversion [4]. Reference calibration adds learner computation; guided rounding adds storage, transport, and state matching; selective precision adds work in the protected region. Table 4 relates these dependencies to the information each method needs.

Token throughput also depends on the realized workload. A policy that produces shorter responses can appear faster without improving execution per token; one that produces longer responses can incur more work despite a faster kernel. Memory savings can increase concurrency, which changes the workload seen by attention and compute kernels. Serving research such as PagedAttention provides background for why cache management and scheduling affect observed throughput [23], but serving speed alone does not establish RL learning efficiency.

Table 4: Information and implementation requirements that distinguish the reviewed approaches. Entries describe method interfaces rather than measured overhead rankings.

Approach	Information or execution dependency	Question for the system comparison
TIS / AIS / CIS	Behavior probabilities and learner/proximal probabilities	Are the stored probabilities those of the actual sampling distribution?
Score Centering	A behavior-weighted score expectation, exact or approximated	What distributional information is returned, and what approximation is used?
Aligned QAT	A learner path corresponding to rollout quantization	Which kernels and numerical interfaces remain different?
Guided rounding	Realized rollout-side quantization information	How are states stored, transferred, and matched to training?
Calibrated Clipping [11]	Higher-precision reference ratios at calibration points	Is the reference-pass cost included in training time?
Selective KV precision	Low-bit cache execution with protected positions	Is protection cost bounded, and under which context workload?

KV4-RL provides a concrete example of distinct timing scopes: over 400 iterations on Qwen3-8B-Base, it reports 1.80× cumulative rollout speedup and 1.27× cumulative full-iteration speedup over BF16 rollout [17]. The two quantities describe the same study, yet already differ substantially. Its common-BF16 evaluation separates learned-policy comparison from evaluation precision. These figures illustrate timing scope rather than a comparison with another paper's hardware or workload.

Time or compute to a specified quality level connects these system measurements to the learning objective. Endpoint scores, iteration time, and rollout throughput each supply part of that picture. Recovering time-to-quality additionally requires the training trajectory, especially when an intervention changes response length, learning speed, or checkpoint selection.

6.4. Limitations of Current Evidence

The principal limitation is transfer across experimental settings. Training horizons, model families, tasks, and hardware differ, making controlled ablations within a study more informative than absolute results across unrelated systems. Mathematical reasoning is a recurring evaluation setting; long interactive trajectories and broader task distributions remain less directly covered. Numerical trans-

Table 5: Conditions governing the interpretation of published learning and efficiency results.

Reported outcome	Conditions of comparison	Scope of interpretation
Smaller mismatch	Same parameters, prefixes, decoding rules, metric definition	Measures conditional agreement; stability requires learning evidence.
Stable training	Horizon, seeds, stopping criteria, quality trajectory	Establishes behavior over the observed horizon and budget.
Comparable accuracy	Evaluation precision, tasks, budget, checkpoint selection	Depends on the evaluation policy and checkpoint rule.
Faster rollout	Tokens, lengths, concurrency, hardware, baseline	Measures generation; full training includes other phases.
Lower memory	W/A/KV scope, allocated vs resident memory, batch policy	Throughput also depends on scheduling and concurrency.
Method combination	Joint ablation, estimator definitions, additional cost	Supports the joint rule under the tested conditions.

fer is equally conditional: format, scaling, higher-precision exceptions, and kernels jointly define the execution path.

The survey inherits these limits and those of its curated narrative scope. The source-linked catalog makes the basis of each observation visible, but provides neither exhaustive retrieval nor independent replication. Where a source leaves an implementation or evaluation condition unreported, that uncertainty remains part of the comparison.

Synthesis. Published evidence most clearly supports relationships between mechanisms and outcomes under stated conditions. Linking numerical agreement, stable updates, policy quality, and runtime is the central task of synthesis.

7. Open Problems

The preceding findings point toward a more predictive account of low-precision RL: one that explains when a discrepancy becomes harmful, which intervention is sufficient, and how its benefit survives a change of system or task.

7.1. Predicting Harmful Mismatch

Mean disagreement is an incomplete predictor because learning depends on where error occurs and which update it changes. Confidence-conditioned tails, signed clipping distortions, conditional drift, and directional concentration offer more specific candidates. The unresolved step is to connect these quantities to future learning damage before a task score reveals it.

A predictor of collapse within a horizon need not predict reduced improvement per update or failure to reach a quality threshold. Establishing its target makes the result interpretable. Transfer is the stronger test: a diagnostic that remains predictive after changing the correction or model is more likely to identify a mechanism than a feature of one training recipe. Intervention sensitivity is equally informative. When a control lowers the metric without improving learning, the relationship may be correlational or the control may suppress useful signal along with the risk.

7.2. Composing Numerical and Learning Interventions

Numerical fidelity changes the operating regime of a correction. Conversely, a correction that suppresses updates can change the apparent value of finer quantization. Their joint design therefore depends on marginal benefit: how much additional learning a component preserves after another component has already acted.

This interaction is especially delicate when multiple masks, truncations, or trust bands affect the same examples. An apparently stable combination can owe its behavior to weak learning. A useful account of composition would connect the joint estimator to the signed signal that survives, then relate that signal to quality gained per unit of computation. The information exchanged between endpoints belongs in the same comparison, since a statistically attractive correction may require expensive reference or distributional computations.

7.3. Precision, Staleness, and Long-Horizon Interaction

Partial and asynchronous collection can make the sampling version token-dependent. A probability gap may then combine parameter change with execution mismatch, as Equation (1) makes explicit. Separating their effects requires information about snapshots, replay, or retained cache states that a production system may not store. The smallest behavior record sufficient for reliable correction remains a joint statistical and system question.

Long responses amplify this difficulty. An early numerical perturbation changes the prefixes visited later, even if conditional correction at each observed prefix is accurate. This suggests allocating precision according to position, layer, context, and training phase, with learning benefit as the objective. Local tensor error alone cannot identify where the extra fidelity is worth its cost.

Multi-turn interaction extends the same question beyond token prefixes. Tool observations and environment transitions add state that shapes later behavior and may need to enter the record alongside cache provenance and sampling probabilities. The taxonomy still locates numerical and learning interventions, but evidence from single-response generation leaves their effectiveness in these settings open.

7.4. Generalization and Comparable Evidence

Transfer becomes easier to assess when results identify the learner and sampler paths, actual behavior probabilities, update ratios, collection schedule, evaluation precision, and timing scope. These details explain why two nominally identical precision settings can behave differently. They also distinguish an adopted algorithm from a system's integration contribution and native arithmetic from quantization emulation.

A shared vocabulary would allow results to accumulate across diverse evaluations. Learning trajectories and explicit checkpoint rules connect stability to quality; phase timing connects kernel gains to training cost; joint ablations connect component mechanisms to effective combinations. A universal benchmark is unnecessary for this narrative synthesis, but comparable evidence is essential to turning promising individual designs into transferable principles.

8. Conclusion

Quantized rollout changes the computation that generates reinforcement-learning data, allowing numerical approximation to return through policy updates and future trajectories. Its consequences become clearer when execution mismatch is separated from policy staleness, and conditional correction from changes in the histories that are sampled.

The reviewed methods respond at complementary interfaces. Numerical alignment shares computation; fidelity control preserves sensitive information and parameter changes; estimator correction and update control determine how residual mismatch enters learning. Schedule adaptation adds a broader optimization perspective, with direct low-bit evidence still emerging. Across these approaches, the most useful comparisons connect a named mechanism to the learning signal it preserves and the cost of preserving it.

This perspective shifts attention from reducing a single discrepancy statistic to understanding effective learning under approximate execution. Predictive diagnostics, coherent method composition, and explicit behavior semantics can help the field move from successful individual configurations toward principles that transfer across models, numerical paths, and interaction settings.

A. Surveyed Papers and Section Mapping

Table 6 summarizes all twenty focal contributions, their numerical settings, operative components, and discussion locations. The year denotes first public appearance; the bibliography may cite a later formal publication. Core, supporting, and boundary roles describe their use in this review. A supporting study can still contain direct quantized-rollout experiments. Primary discussion locations appear first, followed by secondary locations in parentheses.

Table 6: Surveyed papers: numerical regimes, method components, and section mapping. Conference and preprint versions count as one contribution.

Work / first year	Numerical regime	Component(s) and contribution	Discussion
Core method and system studies (13)			
FlashRL / TIS [1] 2025	Higher-precision learner; quantized rollout	**B1**: truncated importance correction using recorded behavior probabilities.	5.1 (1.1; 2.1)
Jet-RL [3] 2026	Unified FP8 learner and rollout paths	**A1; A2**: coordinated precision flow and quantization granularity.	4.1 (6.2)
FP8-RL [4] 2026	W8A8 rollout; FP8 attention / KV; FP8 learner explored in v2	**A2; system**: scaling, calibration, and synchronization; adopts TIS/MIS.	4.2 (5.1; 6.2; 6.3)
QuRL [5] 2026	INT8 / FP8 rollout	**B2; A2**: adaptive clipping (ACR) and update-aware quantization (UAQ).	5.2 (3.1; 4.2; 5.4; 6.2)
QaRL [6] 2026	Quantized MoE rollout; aligned learner QAT	**A1; B2**: rollout-aligned QAT and sequence-level trust-band updates (TBPO).	4.1 (5.2; 5.4; 6.2)
AIS [7] 2026	BF16 learner / FP8 rollout; autoregressive and diffusion models	**B1**: adaptive mixture of uncorrected and importance-weighted estimators.	5.1 (6.1; 6.2)
QUADS [8] 2026	NVFP4 MoE rollout; learner weight QAT	**A1; A2**: asymmetric weight QAT and rollout residual activation compensation.	4.1 (4.2; 5.4; 6.2)
ACRL [9] 2026	FP8 rollout; discrepancy-dependent updates	**B2**: advantage-guided probability adjustment regulates discrepancy.	5.2 (3.3; 6.2)
Score Centering [10] 2026	BF16 learner; quantized rollout or policy lag	**B1**: additive score centering; behavior expectation is approximated.	5.1 (2.2; 3.2; 5.4; 6.2)
Full Pipeline FP8 RL [11] 2026	FP8 learner and rollout; calibration against BF16	**B2; system**: reference-calibrated clipping preserves signed contributions.	5.2 (3.2; 6.2; 6.3)
TRIAGE [12] 2026	Native NVFP4 W4A4; MoE analysis	**B2**: direction-selective gating, rebalancing, and bounded local repair.	5.2 (3.3; 6.2)
TRACE [13] 2026	FP4 MoE rollout; guided learner QAT	**A1**: transfers rollout quantization information to guide learner rounding.	4.1 (3.1; 6.2)
KV4-RL [17] 2026	INT4 KV with bounded INT8 protection	**A2; system**: protects sensitive cache positions under retained TIS; generation and optimization alternate.	4.2 (5.1; 6.2; 6.3)
Supporting formulation and general TIM studies (6)			
R3 [15] 2025	MoE routing; general execution mismatch	**A1 context**: replay of discrete routing choices; not uniquely quantization-induced.	3.1 (4.1)
FP16-Mismatch [16] 2025	BF16 / FP16 numerical comparison	**A2 context**: precision-selection evidence; not a low-bit acceleration method.	3.1

Continued on next page

Table 6 continued: Surveyed papers

Work / first year	Numerical regime	Component(s) and contribution	Discussion
MiniRL [14] 2025	Hybrid-engine RL; includes FP8-rollout experiments	**Formulation; B1/B2**: distinguishes behavior, proximal, and learner roles.	2.1 (1.2; 5.1)
Beyond Precision [18] 2026	General TIM; optimization scheduling	**B3**: mismatch-informed learning-rate scheduling; FP4 transfer remains open.	5.3 (3.3)
VeXact / Diagnosing TIM [19] 2026	Matched-execution diagnostic reference	**Diagnosis**: zero-mismatch counterfactual for attribution.	3.4
CIS [20] 2026	BF16 cross-engine mismatch	**B1 context**: confidence-dependent truncation; not direct FP4 validation.	5.1 (3.2)
Exploration boundary case (1)			
QeRL [2] 2025	NVFP4 with LoRA and adaptive noise	**Exploration boundary**: quantization noise can aid exploration in its low-rank setting.	Box 1

B. Source Selection and Version Handling

Construction of this snapshot. The corpus was developed from a seed collection of seventeen papers and expanded with three general TIM studies relevant to the formulation and analysis. Method and scope statements were checked against available primary papers, official publication pages, and arXiv records. This selection supports a focused narrative review; a systematic database search and exhaustive screening were outside its scope.

Inclusion and exclusion. A focal study contributes a direct quantized-rollout or low-precision RL intervention, a mechanism needed to interpret that setting, or a carefully identified boundary case. General quantization and RL background is cited selectively. Papers devoted only to low-precision pretraining, unrelated generation modalities, or alternative rollout acceleration do not become focal entries solely because they use the same precision format.

Versions and names. The catalog records the version used where available and preserves first-publication dates separately from publication venues. QeRL, QuRL, and QaRL use the saved formal publication records in the bibliography. Aliases and changes in presentation are not treated as new contributions. Where an arXiv link is used without a pinned version, the catalog distinguishes that link from a locally reviewed version. A subsequent update should revise both the version record and any claim affected by the revision.

Attribution of evidence. Every method-specific statement is attributed to its primary source. Equations (1), (2), (4), (5), and (6) are explanatory identities under their stated conditions; they are not new empirical findings. Figures and summary tables are original organizing devices. The companion repository provides the focal catalog, bibliography, and source-linked evidence records, together with the manuscript and its revision history. It contains links to third-party papers rather than redistributed paper files.

C. Interpreting New Evidence

A new result first needs a clear account of the policy that generated its data. Parameter versions, numerical paths, decoding transformations, and retained execution state determine how recorded probabilities relate to actual behavior. The learner, proximal, and KL-reference policies then identify the ratios used in correction and optimization.

The component taxonomy locates the operation being changed, while configuration labels describe its precision target, architecture, and collection schedule. This separation makes the remaining approximation visible: conditional rather than response-level correction, truncated weights, partial execution alignment, or an approximated expectation.

Finally, an outcome acquires meaning through its conditions. Numerical fidelity, stability over a horizon, learned-policy quality, and runtime answer different questions. Published combinations provide evidence for their joint rule; an unevaluated combination remains a research hypothesis. Recording these distinctions allows new studies to refine the synthesis as the literature grows.

References

[1] Feng Yao, Liyuan Liu, Dinghuai Zhang, Chengyu Dong, Jingbo Shang, Jianfeng Gao. On the Rollout-Training Mismatch in Modern RL Systems. *OPT 2025: Optimization for Machine Learning*, 2025. https://www.opt-ml.org/papers/2025/paper116.pdf.

[2] Wei Huang, Yi Ge, Shuai Yang, Yicheng Xiao, Huizi Mao, Yujun Lin, Hanrong Ye, Sifei Liu, Ka Chun Cheung, Hongxu Yin, Yao Lu, Xiaojuan Qi, Song Han, Yukang Chen. QeRL: Quantization-enhanced Low-Rank Reinforcement Learning for LLMs. *International Conference on Learning Representations*, 2026. https://proceedings.iclr.cc/paper_files/paper/2026/hash/a15a2ece7f0663d1ba7db91103ac61c9-Abstract-Conference.html.

[3] Haocheng Xi, Charlie Ruan, Peiyuan Liao, Yujun Lin, Han Cai, Yilong Zhao, Shuo Yang, Kurt Keutzer, Song Han, Ligeng Zhu. Jet-RL: Enabling On-Policy FP8 Reinforcement Learning with Unified Training and Rollout Precision Flow. *arXiv preprint*, 2026. https://arxiv.org/abs/2601.14243.

[4] Zhaopeng Qiu, Shuang Yu, Jingqi Zhang, Shuai Zhang, Xue Huang, Jingyi Yang, Junjie Lai. FP8-RL: A Practical and Stable Low-Precision Stack for LLM Reinforcement Learning. *arXiv preprint*, 2026. https://arxiv.org/abs/2601.18150.

[5] Yuhang Li, Reena Elangovan, Xin Dong, Priyadarshini Panda, Brucek Khailany. QuRL: Efficient Reinforcement Learning with Quantized Rollout. *International Conference on Learning Representations*, 2026. https://proceedings.iclr.cc/paper_files/paper/2026/hash/6b4880ced785077159b340710ceb7681-Abstract-Conference.html.

[6] Hao Gu, Hao Wang, Jiacheng Liu, Lujun Li, Qiyuan Zhu, Bei Liu, Binxing Xu, Lei Wang, Xintong Yang, Sida Lin, Sirui Han, Yike Guo. QaRL: Rollout-Aligned Quantization-Aware RL for Fast and Stable Training under Training–Inference Mismatch. *Findings of the Association for Computational Linguistics: ACL 2026*, 2026. https://aclanthology.org/2026.findings-acl.841/.

[7] Jiajun Zhou, Wei Shao, Lingchao Zheng, Yuwei Fan, Ngai Wong. AIS: Adaptive Importance Sampling for Quantized RL. *arXiv preprint*, 2026. https://arxiv.org/abs/2605.13907.

[8] Zhengyang Zhuge, Hao Yu, Xin Wang, Zheng Li, Yizhong Cao, Dayiheng Liu, Jianwei Zhang. QUADS: Stabilizing NVFP4 Reinforcement Learning for MoE via QUantization-error Alignment across Dual Sides. *arXiv preprint*, 2026. https://arxiv.org/abs/2607.15810.

[9] Wenwu Fan, Qihong Lin, Zhijie Xia, Zhuo Zheng, Sihao Wang, Qiang Chen, Liangsheng Zhu. ACRL: Adaptive Control of Training-Inference Discrepancy for Stable Reinforcement Learning. *arXiv preprint*, 2026. https://arxiv.org/abs/2607.24062.

[10] Martin Marek, Max Ryabinin. Score Centering Stabilizes Off-policy Reinforcement Learning. *arXiv preprint*, 2026. https://arxiv.org/abs/2609.20807.

[11] Fanchao Chen, Ziheng Jiang, Ziyun Wei, Zheng Zhong, Du Li, Chi Zhang, Haibin Lin, Shivaram Venkataraman. Towards Full Pipeline FP8 Reinforcement Learning for LLMs. *arXiv preprint*, 2026. https://arxiv.org/abs/2609.22870.

[12] Zhen Li, Shuai Zhang, Yanggan Gu, Yiming Zhang, Yang Yu, Mingfa Feng, Congkai Xie, Shuang Yu, Junjie Lai, Hongxia Yang. TRIAGE: Direction-Aware Mismatch Stabilization of Native NVFP4 Reinforcement Learning. *arXiv preprint*, 2026. https://arxiv.org/abs/2610.07043.

[13] Xin Wang, Hao Yu, Zhengyang Zhuge, Bochao Mao, Zheng Li, Junda Feng, Yuyan Luo, Yi Zhang, Yizhong Cao, Mi Zhang, Dayiheng Liu, Jianwei Zhang. TRACE: Rollout-Guided Quantization-Aware Training for FP4 Reinforcement Learning of MoE Language Models. *arXiv preprint*, 2026. https://arxiv.org/abs/2610.07767.

[14] Chujie Zheng, Kai Dang, Bowen Yu, Mingze Li, Huiqiang Jiang, Junrong Lin, Yuqiong Liu, Hao Lin, Chencan Wu, Feng Hu, An Yang, Jingren Zhou, Junyang Lin. Stabilizing Reinforcement Learning with LLMs: Formulation and Practices. *arXiv preprint*, 2025. https://arxiv.org/abs/2512.01374.

[15] Wenhan Ma, Hailin Zhang, Liang Zhao, Yifan Song, Yudong Wang, Zhifang Sui, Fuli Luo. Stabilizing MoE Reinforcement Learning by Aligning Training and Inference Routers. *arXiv preprint*, 2025. https://arxiv.org/abs/2510.11370.

[16] Penghui Qi, Zichen Liu, Xiangxin Zhou, Tianyu Pang, Chao Du, Wee Sun Lee, Min Lin. Defeating the Training-Inference Mismatch via FP16. *arXiv preprint*, 2025. https://arxiv.org/abs/2510.26788.

[17] Sihwan Park, Sung-Yub Kim, Doohyuk Jang, Eunho Yang. KV4-RL: Unlocking Faster Asynchronous Rollouts for RLVR with INT4 KV Cache Quantization. *arXiv preprint*, 2026. https://arxiv.org/abs/2610.14448.

[18] Yaxiang Zhang, Yingru Li, Jiacai Liu, Jiawei Xu, Ziniu Li, Qian Liu, Haoyuan Li. Beyond Precision: Training-Inference Mismatch is an Optimization Problem and Simple LR Scheduling Fixes It. *arXiv preprint*, 2026. https://arxiv.org/abs/2602.01826.

[19] Tianle Zhong, Neiwen Ling, Yifan Pi, Zijun Wei, Tianshu Yu, Geoffrey Fox, Peng Wu, Xiao Yu. Diagnosing Training Inference Mismatch in LLM Reinforcement Learning via a Zero-Mismatch Reference. *arXiv preprint*, 2026. https://arxiv.org/abs/2605.14220.

[20] Tianrun Yu, Kaixiang Zhao, Shangzhe Li, Yuxiao Yang, Porter Jenkins, Weitong Zhang, Taylor W. Killian. Rethinking Training-Inference Mismatch in LLM Reinforcement Learning: Where It Arises and How to Correct It. *arXiv preprint*, 2026. https://arxiv.org/abs/2609.32444.

[21] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, Oleg Klimov. Proximal Policy Optimization Algorithms. *arXiv preprint*, 2017. https://arxiv.org/abs/1707.06347.

[22] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, Daya Guo. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint*, 2024. https://arxiv.org/abs/2402.03300.

[23] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, Ion Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. *arXiv preprint*, 2023. https://arxiv.org/abs/2309.06180.

[24] Zhiwei Hao, Jianyuan Guo, Li Shen, Yong Luo, Han Hu, Guoxia Wang, Dianhai Yu, Yonggang Wen, Dacheng Tao. Low-Precision Training of Large Language Models: Methods, Challenges, and Opportunities. *arXiv preprint*, 2025. https://arxiv.org/abs/2505.01043.

[25] Rohan Surana, Gagan Mundada, Xunyi Jiang, Chuhan Wang, Zhenwei Tang, Difan Jiao, Zihan Huang, Yuxin Xiong, Junda Wu, Sheldon Yu, Xintong Li, Raghav Jain, Nikki Kuang, Sizhe Zhou, Bowen Jin, Zhendong Chu, Tong Yu, Ryan Rossi, Kuan-Hao Huang, Jingbo Shang, Jiawei Han, Julian McAuley. Generate, Filter, Control, Replay: A Comprehensive Survey of Rollout Strategies for LLM Reinforcement Learning. *arXiv preprint*, 2026. https://arxiv.org/abs/2605.02913.

[26] Kaiyan Zhang, Yuxin Zuo, Bingxiang He, Youbang Sun, Runze Liu, Che Jiang, Yuchen Fan, Kai Tian, Guoli Jia, Pengfei Li, Yu Fu, Xingtai Lv, Yuchen Zhang, Sihang Zeng, Shang Qu, Haozhan Li, Shijie Wang, Yuru Wang, Xinwei Long, Fangfu Liu, Xiang Xu, Jiaze Ma, Xuekai Zhu, Ermo Hua, Yihao Liu, Zonglin Li, Huayu Chen, Xiaoye Qu, Yafu Li, Weize Chen, Zhenzhao Yuan, Junqi Gao, Dong Li, Zhiyuan Ma, Ganqu Cui, Zhiyuan Liu, Biqing Qi, Ning Ding, Bowen Zhou. A Survey of Reinforcement Learning for Large Reasoning Models. *arXiv preprint*, 2025. https://arxiv.org/abs/2509.08827.